Lab Report

Lab Sheet-09

Smart Digital Expense Tracker

with Budget Prediction

BCA I Semester (Session 2026-2027)

Course: BCA111P

Name: _____

Roll No.: _____

Submitted to: _____

Date: _____

GitHub Repository

https://github.com/deva308/smart-expense-tracker

1. Objective

To develop a smart expense management system in C++ that helps users record daily expenses, categorize spending, monitor a monthly budget and predict whether that budget will be exceeded.

Problem Statement

Many students spend money daily but do not track their expenses. This project gives an easy way to manage expenses and alerts users before they exceed their monthly budget.

Modules

1. User Registration: creates an account and saves it in users.txt.
2. Login System: checks the username and password, with 3 attempts.
3. Add Daily Expense: stores the date, category, amount and note.
4. View Expense History: shows all expenses sorted by date.
5. Category-wise Expense: shows the total and percentage for each category.
6. Monthly Expense Report: shows a monthly summary and saves it to a file.
7. Budget Setting: sets the monthly budget.
8. Budget Warning: warns at 80% use and alerts when the budget is crossed.
9. Expense Prediction: estimates the month-end total.
10. Search Expense: finds expenses by date, category or a word in the note.
11. Delete Expense: removes an expense by ID after confirmation.
12. File Saving: keeps all data in text files between runs.

Concepts Used

Variables, loops, arrays, structures, functions, file handling, strings, searching (linear search) and sorting (bubble sort).

2. Algorithm and Logic

Main Program

1. Start the program and show the menu: Register, Login, Exit.
2. If the user registers, validate the username and password, then save them in users.txt.
3. If the user logs in, compare the details with the saved users, allowing 3 attempts.
4. After login, load that user's expenses and budget from their files.
5. Show the expense menu and run the feature the user selects.
6. Save the file after every change, so no data is lost.
7. Repeat until the user logs out, then return to the main menu.
8. Stop when the user selects Exit.

Budget Warning

```
percentUsed = spent * 100 / monthlyBudget
if spent > monthlyBudget        -> show ALERT with the extra amount
else if percentUsed >= 80       -> show WARNING with the amount left
else                            -> show the amount left
```

Expense Prediction

```
averagePerDay  = spentSoFar / today's day number
predictedTotal = averagePerDay * days in the month
if predictedTotal > monthlyBudget
    show that the budget will be exceeded
    if days are left and spentSoFar < monthlyBudget
        safeDailySpend = (monthlyBudget - spentSoFar) / daysLeft
else
    show the expected saving
```

The prediction assumes the user keeps spending at the same daily average for the rest of the month, so it is an estimate and not an exact amount.

Sorting and Searching

Each date is converted into one number (year * 10000 + month * 100 + day), and bubble sort compares these numbers to arrange expenses from oldest to newest. Search and delete use linear search, which checks every expense one by one.

3. Sample Input and Output

These results come from a test run of the compiled program, with a monthly budget of Rs. 5000.

Sample 1: Budget warning after adding an expense

Input: Date 8/11/2026, Category Education, Amount 600, Note Books

Output:

```
Expense added.
Budget: Rs. 5000.00 | Spent: Rs. 4600.00 | Used: 92.00%
WARNING: Only Rs. 400.00 left this month.
```

Sample 2: Over budget alert

Input: Date 9/11/2026, Category Bills, Amount 700, Note Recharge

Output:

```
Expense added.
Budget: Rs. 5000.00 | Spent: Rs. 5300.00 | Used: 106.00%
ALERT: You are over budget by Rs. 300.00!
```

Sample 3: Expense history

Input: Menu option 2

Output:

```
ID   Date         Category         Amount  Note
------------------------------------------------------------
4    30/10/2026   Shopping          900.00  Shoes
2    2/11/2026    Travel           2500.00  Bus pass
1    5/11/2026    Food             1500.00  College canteen / lunch
3    8/11/2026    Education         600.00  Books
5    9/11/2026    Bills             700.00  Recharge
Total: Rs. 6200.00 in 5 expense(s)
```

Sample 4: Category-wise expense

Input: Menu option 3

Output:

```
Food            1500.00    24.19%
Travel          2500.00    40.32%
Education        600.00     9.68%
Shopping         900.00    14.52%
Bills            700.00    11.29%
Other              0.00     0.00%
Total           6200.00
```

Sample 5: Monthly report

Input: Menu option 4, Year 2026, Month 11

Output:

```
Monthly Report for ram_1 - 11/2026
ID   Date         Category         Amount  Note
------------------------------------------------------------
2    2/11/2026    Travel          2500.00  Bus pass
1    5/11/2026    Food            1500.00  College canteen / lunch
3    8/11/2026    Education        600.00  Books
5    9/11/2026    Bills            700.00  Recharge
------------------------------------------------------------
Food        Rs. 1500.00
Travel      Rs. 2500.00
Education   Rs. 600.00
Bills       Rs. 700.00
Number of expenses: 4
Highest expense:    Rs. 2500.00
Total spent:        Rs. 5300.00
Monthly budget:     Rs. 5000.00
Balance:            Rs. -300.00
Report saved to report_ram_1_11_2026.txt
```

Sample 6: Expense prediction

Input: Menu option 7, today's date 10/11/2026 (after deleting expense ID 2)

Output:

```
Spent so far:        Rs. 2800.00
Average per day:     Rs. 280.00
Predicted month end: Rs. 8400.00
Monthly budget:      Rs. 5000.00
PREDICTION: Budget will be exceeded by Rs. 3400.00.
To stay in budget, spend at most Rs. 110.00 per day.
```

Sample 7: Search by word in note

Input: Menu option 8, Search type 3, Word BUS

Output:

```
ID   Date         Category         Amount  Note
------------------------------------------------------------
2    2/11/2026    Travel          2500.00  Bus pass
1 expense(s) found. Total: Rs. 2500.00
```

Sample 8: Invalid input handling

Input: Date 31/11/2026, then Amount abc

Output:

```
Please enter a number from 1 to 30.
Please enter an amount greater than 0.
```

4. Conclusion

The Smart Digital Expense Tracker was developed, compiled and tested successfully. It records daily expenses, produces category-wise and monthly reports, warns the user about the budget and predicts the month-end total. All data stays saved in text files.

This project gave practical experience of file handling, structures, arrays, functions, searching, sorting, menu-driven programming and report generation on a real-world problem.

Limitations

- Passwords are stored as plain text, so the program should not be used with real passwords.
- It stores up to 50 users and 500 expenses per user.
- The prediction uses a simple daily average.

Future Enhancements

- AI-based spending prediction
- Mobile app
- QR bill scanner
- Cloud backup

5. Source Code (expense_tracker.cpp)

```
/*
    Smart Digital Expense Tracker with Budget Prediction
    BCA I Semester (Session 2026-2027)

    A menu-driven program that lets a user register, log in, record daily
    expenses, view reports, set a monthly budget and predict whether the
    budget will be exceeded. All data is saved in text files, so nothing
    is lost when the program closes.

    Compile: g++ expense_tracker.cpp -o expense_tracker
*/

#include <iostream>
#include <fstream>
#include <iomanip>
#include <limits>
#include <string>
#include <cctype>
#include <cstdlib>

using namespace std;

const int MAX_USERS = 50;
const int MAX_EXPENSES = 500;
const int CATEGORY_COUNT = 6;
const double WARNING_PERCENT = 80.0;
const string USERS_FILE = "users.txt";
const string CATEGORIES[CATEGORY_COUNT] = {"Food", "Travel", "Education",
                                           "Shopping", "Bills", "Other"};

struct User {
    string username;
    string password;
};

struct Expense {
    int id;
    int day;
    int month;
    int year;
    double amount;
    string category;
    string note;
};

// Data of the logged-in user, kept in memory while the program runs.
Expense expenses[MAX_EXPENSES];
int expenseCount = 0;
double monthlyBudget = 0;
string currentUser = "";

// ---------- Input helpers ----------

// Removes leftover characters so the next input starts on a fresh line.
void clearInputLine() {
    if (cin.eof()) {
        cout << "\nInput ended. Exiting program.\n";
        exit(0);
    }
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

// Keeps asking until the user types a whole number inside the given range.
int readInt(string prompt, int minValue, int maxValue) {
    int value;
    while (true) {
        cout << prompt;
        if (cin >> value && value >= minValue && value <= maxValue) {
```

```
            clearInputLine();
            return value;
        }
        clearInputLine();
        cout << "Please enter a number from " << minValue << " to " << maxValue << ".\n";
    }
}

// Keeps asking until the user types a number greater than zero.
double readPositiveAmount(string prompt) {
    double value;
    while (true) {
        cout << prompt;
        if (cin >> value && value > 0) {
            clearInputLine();
            return value;
        }
        clearInputLine();
        cout << "Please enter an amount greater than 0.\n";
    }
}

// Reads a full line. The | character is replaced because it separates
// fields in the data files.
string readLine(string prompt) {
    string text;
    cout << prompt;
    if (!getline(cin, text)) {
        clearInputLine();
    }
    for (int i = 0; i < (int)text.length(); i++) {
        if (text[i] == '|') {
            text[i] = '/';
        }
    }
    return text;
}

string toLowerCase(string text) {
    for (int i = 0; i < (int)text.length(); i++) {
        text[i] = tolower((unsigned char)text[i]);
    }
    return text;
}

// A username becomes part of a file name, so only letters, digits and _ are allowed.
bool isValidUsername(string name) {
    if (name.length() < 3 || name.length() > 20) {
        return false;
    }
    for (int i = 0; i < (int)name.length(); i++) {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_') {
            return false;
        }
    }
    return true;
}

// ---------- Date helpers ----------

bool isLeapYear(int year) {
    return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
}

int daysInMonth(int month, int year) {
    int days[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    if (month == 2 && isLeapYear(year)) {
        return 29;
    }
    return days[month - 1];
}

// Reads a valid calendar date, so 31/04 or 30/02 is rejected.
```

```
void readDate(int &day, int &month, int &year) {
    year = readInt("Year (2000-2100): ", 2000, 2100);
    month = readInt("Month (1-12): ", 1, 12);
    day = readInt("Day (1-" + to_string(daysInMonth(month, year)) + "): ",
                  1, daysInMonth(month, year));
}

// Turns a date into one number (20261105) so two dates are easy to compare.
int dateKey(Expense expense) {
    return expense.year * 10000 + expense.month * 100 + expense.day;
}

// ---------- File saving ----------

string expenseFileName() {
    return "expenses_" + currentUser + ".txt";
}

string budgetFileName() {
    return "budget_" + currentUser + ".txt";
}

// Each line in the file: id|day|month|year|amount|category|note
void saveExpenses() {
    ofstream file(expenseFileName());
    if (!file) {
        cout << "Error: could not save expenses.\n";
        return;
    }
    file << fixed << setprecision(2);
    for (int i = 0; i < expenseCount; i++) {
        file << expenses[i].id << '|' << expenses[i].day << '|' << expenses[i].month << '|'
             << expenses[i].year << '|' << expenses[i].amount << '|'
             << expenses[i].category << '|' << expenses[i].note << '\n';
    }
}

void loadExpenses() {
    expenseCount = 0;
    ifstream file(expenseFileName());
    Expense expense;
    char separator;
    while (expenseCount < MAX_EXPENSES &&
           file >> expense.id >> separator >> expense.day >> separator >> expense.month
                >> separator >> expense.year >> separator >> expense.amount >> separator) {
        getline(file, expense.category, '|');
        getline(file, expense.note);
        expenses[expenseCount] = expense;
        expenseCount++;
    }
}

void saveBudget() {
    ofstream file(budgetFileName());
    if (!file) {
        cout << "Error: could not save budget.\n";
        return;
    }
    file << fixed << setprecision(2) << monthlyBudget << '\n';
}

void loadBudget() {
    monthlyBudget = 0;
    ifstream file(budgetFileName());
    file >> monthlyBudget;
}

// ---------- Registration and login ----------

int loadUsers(User users[]) {
    int count = 0;
    ifstream file(USERS_FILE);
```

```
    string line;
    while (count < MAX_USERS && getline(file, line)) {
        int separatorIndex = line.find('|');
        if (separatorIndex > 0) {
            users[count].username = line.substr(0, separatorIndex);
            users[count].password = line.substr(separatorIndex + 1);
            count++;
        }
    }
    return count;
}

// Linear search through the users. Returns the index, or -1 if not found.
int findUser(User users[], int count, string username) {
    for (int i = 0; i < count; i++) {
        if (toLowerCase(users[i].username) == toLowerCase(username)) {
            return i;
        }
    }
    return -1;
}

void registerUser() {
    User users[MAX_USERS];
    int count = loadUsers(users);

    cout << "\n--- User Registration ---\n";
    if (count >= MAX_USERS) {
        cout << "User limit reached. Cannot register more users.\n";
        return;
    }

    string username = readLine("Choose a username (3-20 letters, digits or _): ");
    if (!isValidUsername(username)) {
        cout << "Invalid username.\n";
        return;
    }
    if (findUser(users, count, username) != -1) {
        cout << "This username is already taken.\n";
        return;
    }

    string password = readLine("Choose a password (at least 4 characters): ");
    if (password.length() < 4) {
        cout << "Password is too short.\n";
        return;
    }

    ofstream file(USERS_FILE, ios::app);
    if (!file) {
        cout << "Error: could not save user.\n";
        return;
    }
    file << username << '|' << password << '\n';
    cout << "Registration successful. You can log in now.\n";
}

// Gives the user 3 attempts. Returns true when the login succeeds.
bool loginUser() {
    User users[MAX_USERS];
    int count = loadUsers(users);

    cout << "\n--- Login ---\n";
    for (int attempt = 1; attempt <= 3; attempt++) {
        string username = readLine("Username: ");
        string password = readLine("Password: ");
        int index = findUser(users, count, username);
        if (index != -1 && users[index].password == password) {
            currentUser = users[index].username;
            cout << "Welcome, " << currentUser << "!\n";
            return true;
        }
        cout << "Wrong username or password. Attempts left: " << 3 - attempt << "\n";
```

```
    }
    return false;
}

// ---------- Calculations ----------

double monthTotal(int month, int year) {
    double total = 0;
    for (int i = 0; i < expenseCount; i++) {
        if (expenses[i].month == month && expenses[i].year == year) {
            total += expenses[i].amount;
        }
    }
    return total;
}

// ---------- Display helpers ----------

void printExpenseHeader(ostream &out) {
    out << left << setw(5) << "ID" << setw(13) << "Date" << setw(12) << "Category"
        << right << setw(11) << "Amount" << "  " << left << "Note\n";
    out << string(60, '-') << "\n";
}

void printExpense(ostream &out, Expense expense) {
    string date = to_string(expense.day) + "/" + to_string(expense.month) + "/" +
                  to_string(expense.year);
    out << left << setw(5) << expense.id << setw(13) << date << setw(12) << expense.category
        << right << setw(11) << expense.amount << "  " << left << expense.note << "\n";
}

// Bubble sort: oldest expense first.
void sortExpensesByDate() {
    for (int i = 0; i < expenseCount - 1; i++) {
        for (int j = 0; j < expenseCount - 1 - i; j++) {
            if (dateKey(expenses[j]) > dateKey(expenses[j + 1])) {
                Expense temp = expenses[j];
                expenses[j] = expenses[j + 1];
                expenses[j + 1] = temp;
            }
        }
    }
}

// ---------- Budget warning ----------

void showBudgetStatus(int month, int year) {
    if (monthlyBudget <= 0) {
        cout << "No budget set yet. Use the Set Budget option.\n";
        return;
    }
    double spent = monthTotal(month, year);
    double percentUsed = spent * 100 / monthlyBudget;


    cout << "Budget: Rs. " << monthlyBudget << " | Spent: Rs. " << spent
         << " | Used: " << percentUsed << "%\n";
    if (spent > monthlyBudget) {
        cout << "ALERT: You are over budget by Rs. " << spent - monthlyBudget << "!\n";
    } else if (percentUsed >= WARNING_PERCENT) {
        cout << "WARNING: Only Rs. " << monthlyBudget - spent << " left this month.\n";
    } else {
        cout << "You are within budget. Rs. " << monthlyBudget - spent << " left.\n";
    }
}

// ---------- Menu features ----------

void addExpense() {
    cout << "\n--- Add Daily Expense ---\n";
    if (expenseCount >= MAX_EXPENSES) {
        cout << "Storage is full. Delete an expense first.\n";
        return;
```

```
    }

    Expense expense;
    int highestId = 0;
    for (int i = 0; i < expenseCount; i++) {
        if (expenses[i].id > highestId) {
            highestId = expenses[i].id;
        }
    }
    expense.id = highestId + 1;

    readDate(expense.day, expense.month, expense.year);
    for (int i = 0; i < CATEGORY_COUNT; i++) {
        cout << i + 1 << ". " << CATEGORIES[i] << "\n";
    }
    expense.category = CATEGORIES[readInt("Category: ", 1, CATEGORY_COUNT) - 1];
    expense.amount = readPositiveAmount("Amount (Rs.): ");
    expense.note = readLine("Note: ");

    expenses[expenseCount] = expense;
    expenseCount++;
    saveExpenses();

    cout << "Expense added.\n";
    showBudgetStatus(expense.month, expense.year);
}

void viewHistory() {
    cout << "\n--- Expense History ---\n";
    if (expenseCount == 0) {
        cout << "No expenses recorded yet.\n";
        return;
    }
    sortExpensesByDate();
    double total = 0;
    printExpenseHeader(cout);
    for (int i = 0; i < expenseCount; i++) {
        printExpense(cout, expenses[i]);
        total += expenses[i].amount;
    }
    cout << "Total: Rs. " << total << " in " << expenseCount << " expense(s)\n";
}

void categoryWiseExpense() {
    cout << "\n--- Category-wise Expense ---\n";
    double categoryTotals[CATEGORY_COUNT] = {0};
    double grandTotal = 0;

    for (int i = 0; i < expenseCount; i++) {
        for (int j = 0; j < CATEGORY_COUNT; j++) {
            if (expenses[i].category == CATEGORIES[j]) {
                categoryTotals[j] += expenses[i].amount;
                grandTotal += expenses[i].amount;
            }
        }
    }
    if (grandTotal == 0) {
        cout << "No expenses recorded yet.\n";
        return;
    }
    for (int i = 0; i < CATEGORY_COUNT; i++) {
        cout << left << setw(12) << CATEGORIES[i] << right << setw(11) << categoryTotals[i]
             << setw(9) << categoryTotals[i] * 100 / grandTotal << "%\n";
    }
    cout << left << setw(12) << "Total" << right << setw(11) << grandTotal << "\n";
}

// Writes the report to any output stream, so one function serves the
// screen and the saved report file.
void writeMonthlyReport(ostream &out, int month, int year) {
    double categoryTotals[CATEGORY_COUNT] = {0};
    double highest = 0;
    int count = 0;
```

```
    out << "Monthly Report for " << currentUser << " - " << month << "/" << year << "\n";
    printExpenseHeader(out);
    for (int i = 0; i < expenseCount; i++) {
        if (expenses[i].month == month && expenses[i].year == year) {
            printExpense(out, expenses[i]);
            count++;
            if (expenses[i].amount > highest) {
                highest = expenses[i].amount;
            }
            for (int j = 0; j < CATEGORY_COUNT; j++) {
                if (expenses[i].category == CATEGORIES[j]) {
                    categoryTotals[j] += expenses[i].amount;
                }
            }
        }
    }

    double total = monthTotal(month, year);
    out << string(60, '-') << "\n";
    for (int i = 0; i < CATEGORY_COUNT; i++) {
        if (categoryTotals[i] > 0) {
            out << left << setw(12) << CATEGORIES[i] << "Rs. " << categoryTotals[i] << "\n";
        }
    }
    out << "Number of expenses: " << count << "\n";
    out << "Highest expense:    Rs. " << highest << "\n";
    out << "Total spent:        Rs. " << total << "\n";
    if (monthlyBudget > 0) {
        out << "Monthly budget:     Rs. " << monthlyBudget << "\n";
        out << "Balance:            Rs. " << monthlyBudget - total << "\n";
    }
}

void monthlyReport() {
    cout << "\n--- Monthly Expense Report ---\n";
    int year = readInt("Year (2000-2100): ", 2000, 2100);
    int month = readInt("Month (1-12): ", 1, 12);

    if (monthTotal(month, year) == 0) {
        cout << "No expenses found for this month.\n";
        return;
    }

    sortExpensesByDate();
    writeMonthlyReport(cout, month, year);

    string fileName = "report_" + currentUser + "_" + to_string(month) + "_" +
                      to_string(year) + ".txt";
    ofstream file(fileName);
    if (file) {
        file << fixed << setprecision(2);
        writeMonthlyReport(file, month, year);
        cout << "Report saved to " << fileName << "\n";
    } else {
        cout << "Error: could not save report file.\n";
    }
}

void setBudget() {
    cout << "\n--- Budget Setting ---\n";
    if (monthlyBudget > 0) {
        cout << "Current budget: Rs. " << monthlyBudget << "\n";
    }
    monthlyBudget = readPositiveAmount("New monthly budget (Rs.): ");
    saveBudget();
    cout << "Budget saved.\n";
}

void budgetWarning() {
    cout << "\n--- Budget Warning ---\n";
    int year = readInt("Year (2000-2100): ", 2000, 2100);
    int month = readInt("Month (1-12): ", 1, 12);
```

```
    showBudgetStatus(month, year);
}

// Prediction logic: average spend per day so far, multiplied by the
// number of days in the month, gives the expected month-end total.
void predictExpense() {
    cout << "\n--- Expense Prediction ---\n";
    if (monthlyBudget <= 0) {
        cout << "Set a monthly budget first.\n";
        return;
    }

    int day, month, year;
    cout << "Enter today's date.\n";
    readDate(day, month, year);

    double spentSoFar = 0;
    for (int i = 0; i < expenseCount; i++) {
        if (expenses[i].month == month && expenses[i].year == year && expenses[i].day <= day) {
            spentSoFar += expenses[i].amount;
        }
    }
    if (spentSoFar == 0) {
        cout << "No expenses this month yet, so there is nothing to predict from.\n";
        return;
    }

    int totalDays = daysInMonth(month, year);
    int daysLeft = totalDays - day;
    double averagePerDay = spentSoFar / day;
    double predictedTotal = averagePerDay * totalDays;

    cout << "Spent so far:        Rs. " << spentSoFar << "\n";
    cout << "Average per day:     Rs. " << averagePerDay << "\n";
    cout << "Predicted month end: Rs. " << predictedTotal << "\n";
    cout << "Monthly budget:      Rs. " << monthlyBudget << "\n";

    if (predictedTotal > monthlyBudget) {
        cout << "PREDICTION: Budget will be exceeded by Rs. "
             << predictedTotal - monthlyBudget << ".\n";
        if (daysLeft > 0 && spentSoFar < monthlyBudget) {
            cout << "To stay in budget, spend at most Rs. "
                 << (monthlyBudget - spentSoFar) / daysLeft << " per day.\n";
        }
    } else {
        cout << "PREDICTION: You will stay within budget. Expected saving: Rs. "
             << monthlyBudget - predictedTotal << ".\n";
    }
}

void searchExpense() {
    cout << "\n--- Search Expense ---\n";
    cout << "1. By date\n2. By category\n3. By word in note\n";
    int choice = readInt("Search type: ", 1, 3);

    int day = 0, month = 0, year = 0;
    string keyword = "";
    if (choice == 1) {
        readDate(day, month, year);
    } else if (choice == 2) {
        for (int i = 0; i < CATEGORY_COUNT; i++) {
            cout << i + 1 << ". " << CATEGORIES[i] << "\n";
        }
        keyword = CATEGORIES[readInt("Category: ", 1, CATEGORY_COUNT) - 1];
    } else {
        keyword = toLowerCase(readLine("Word to search: "));
    }

    // Linear search: check every expense against the chosen condition.
    int found = 0;
    double total = 0;
    for (int i = 0; i < expenseCount; i++) {
        bool matches = false;
```

```
        if (choice == 1) {
            matches = expenses[i].day == day && expenses[i].month == month &&
                      expenses[i].year == year;
        } else if (choice == 2) {
            matches = expenses[i].category == keyword;
        } else {
            matches = toLowerCase(expenses[i].note).find(keyword) != string::npos;
        }

        if (matches) {
            if (found == 0) {
                printExpenseHeader(cout);
            }
            printExpense(cout, expenses[i]);
            found++;
            total += expenses[i].amount;
        }
    }

    if (found == 0) {
        cout << "No matching expense found.\n";
    } else {
        cout << found << " expense(s) found. Total: Rs. " << total << "\n";
    }
}

void deleteExpense() {
    cout << "\n--- Delete Expense ---\n";
    if (expenseCount == 0) {
        cout << "No expenses to delete.\n";
        return;
    }

    int id = readInt("Expense ID to delete (see View History): ", 1, 1000000);
    int index = -1;
    for (int i = 0; i < expenseCount; i++) {
        if (expenses[i].id == id) {
            index = i;
        }
    }
    if (index == -1) {
        cout << "No expense has this ID.\n";
        return;
    }

    printExpenseHeader(cout);
    printExpense(cout, expenses[index]);
    string answer = readLine("Delete this expense? (y/n): ");
    if (answer != "y" && answer != "Y") {
        cout << "Nothing deleted.\n";
        return;
    }

    // Shift the later expenses one place left to fill the gap.
    for (int i = index; i < expenseCount - 1; i++) {
        expenses[i] = expenses[i + 1];
    }
    expenseCount--;
    saveExpenses();
    cout << "Expense deleted.\n";
}

// ---------- Menus ----------

void userMenu() {
    loadExpenses();
    loadBudget();

    int choice;
    do {
        cout << "\n===== Expense Tracker: " << currentUser << " =====\n";
        cout << "1. Add Daily Expense\n";
        cout << "2. View Expense History\n";
```

```
        cout << "3. Category-wise Expense\n";
        cout << "4. Monthly Expense Report\n";
        cout << "5. Set Monthly Budget\n";
        cout << "6. Budget Warning\n";
        cout << "7. Expense Prediction\n";
        cout << "8. Search Expense\n";
        cout << "9. Delete Expense\n";
        cout << "0. Logout\n";
        choice = readInt("Enter choice: ", 0, 9);

        switch (choice) {
            case 1: addExpense(); break;
            case 2: viewHistory(); break;
            case 3: categoryWiseExpense(); break;
            case 4: monthlyReport(); break;
            case 5: setBudget(); break;
            case 6: budgetWarning(); break;
            case 7: predictExpense(); break;
            case 8: searchExpense(); break;
            case 9: deleteExpense(); break;
            case 0: cout << "Logged out.\n"; break;
        }
    } while (choice != 0);

    currentUser = "";
}

int main() {
    cout << fixed << setprecision(2);

    int choice;
    do {
        cout << "\n===== Smart Digital Expense Tracker =====\n";
        cout << "1. Register\n";
        cout << "2. Login\n";
        cout << "0. Exit\n";
        choice = readInt("Enter choice: ", 0, 2);

        if (choice == 1) {
            registerUser();
        } else if (choice == 2) {
            if (loginUser()) {
                userMenu();
            } else {
                cout << "Too many failed attempts.\n";
            }
        }
    } while (choice != 0);

    cout << "Thank you for using Smart Digital Expense Tracker.\n";
    return 0;
}
```